Problem Set Solutions: Seminar 10

Fenwick Tree

MIT-style solution format

Algorithms and Data Structures I

June 29, 2026

Course:	Algorithms and Data Structures I
Topic:	Fenwick trees, prefix decompositions, range updates, multidimensional sums
Style:	Full problem-set solutions with algorithms, proofs, and complexity bounds

Why this matters. A Fenwick tree is a compact way to encode a canonical decomposition of prefixes into power-of-two aligned blocks. Once this decomposition is understood, many apparently different tasks become the same algebraic trick: store the right block aggregate, update exactly the blocks containing a point, and reduce arbitrary ranges by inclusion-exclusion. This sheet develops the one-dimensional tree, order-statistic search by prefix sums, sparse two-sided variants, modular products with zeros, and the standard lifting to two and three dimensions.

Contents

Conventions. Unless stated otherwise, Fenwick definitions use zero-indexed arrays. Put

$$f(i) = i \,\&\, (i+1), \qquad g(i) = i \,|\, (i+1).$$

The block represented by index i in the direct Fenwick tree is $[f(i), i]$. Logs are base 2.

Problem 1. Fixed points of $f(i) = i \,\&\, (i+1)$

Describe all i such that $f(i) = i$.

Solution.

Answer. Exactly the even nonnegative integers.

Correctness. Write the binary representation of i. Adding 1 changes the last block of trailing ones into zeros and turns the preceding zero into one. Therefore the bitwise operation

$$i \,\&\, (i+1)$$

keeps all bits before the trailing block of ones and replaces the whole trailing block of ones by zeros. Thus $f(i)$ is obtained from i by zeroing the trailing block of ones.

Hence $f(i) = i$ if and only if i has no trailing block of ones, equivalently the least significant bit is 0. That is exactly the condition that i is even. The value $i = 0$ is included.

Running time. This is a bit characterization; no algorithmic work is required.

Problem 2. When does $g(i) = i \,|\, (i+1)$ equal $i+1$?

Describe all i such that $g(i) = i + 1$.

Solution.

Answer. Again, exactly the even nonnegative integers.

Correctness. The expression $g(i) = i \,|\, (i+1)$ sets the least significant zero bit of i to one and leaves all more significant bits unchanged. If i is even, its least significant bit is already the least significant zero bit, so

$$i + 1$$

only changes that final zero to one. Therefore $i \,|\, (i+1) = i + 1$.

If i is odd, the least significant bit is one. Then adding 1 carries through a nonempty trailing block of ones and changes a more significant zero bit. The bitwise OR with i preserves the old trailing ones and also sets that higher zero bit, so the result is strictly larger than $i + 1$. Therefore equality is impossible for odd i.

Running time. Constant-time bit reasoning.

Problem 3. Correctness of the basic Fenwick tree

Given $a_0, a_1, \ldots, a_{n-1}$, define

$$s_i = \sum_{j=f(i)}^{i} a_j.$$

Prove that the standard procedures for prefix sum and point update are correct and take $O(\log n)$ time.

Solution.

Idea. The array s stores sums over canonical blocks $[f(i), i]$. The procedure `get(pos)` decomposes the prefix $[0, pos]$ into disjoint such blocks from right to left. The procedure `update(pos,delta)` visits exactly the stored blocks that contain pos.

Algorithm. The procedures are:

```
int get(int pos) {              // returns a[0] + ... + a[pos]
    int ans = 0;
    for (int i = pos; i >= 0; i = f(i) - 1)
        ans += s[i];
    return ans;
}

void update(int pos, int delta) { // a[pos] += delta
    for (int i = pos; i < n; i = g(i))
        s[i] += delta;
}
```

Correctness. For `get`, suppose the current index is i. The term s_i is exactly the sum on the block $[f(i), i]$. The next index is $f(i) - 1$, so the next block ends immediately before the current block begins. Hence the loop adds disjoint adjacent blocks. After finitely many iterations the right endpoints move from pos down to -1, so the union of all added blocks is exactly $[0, pos]$. Therefore `get(pos)` returns the prefix sum.

For `update`, the value a_{pos} contributes to s_i if and only if

$$f(i) \le pos \le i.$$

The sequence

$$pos,\ g(pos),\ g(g(pos)), \ldots$$

visits exactly those indices i whose canonical block contains pos. This is the duality behind Fenwick trees: moving by g repeatedly sets the next zero bit to one, thereby enumerating all larger aligned blocks that cover the fixed position pos. Therefore adding δ to precisely those s_i and to no others preserves the invariant $s_i = \sum_{j=f(i)}^{i} a_j$.

Running time. Each step of `get` removes at least one trailing block of ones from the current index; each step of `update` sets at least one zero bit. Thus each loop has $O(\log n)$ iterations, and both operations take $O(\log n)$ time.

Problem 4. Point increases and prefix maxima

An array $a_1, \ldots, a_n$ receives two types of queries: increase one element a_{pos} by $x \geq 0$, and report the maximum on a prefix. Process all queries in $O((n+q)\log n)$.

Solution.

Idea. Replace block sums by block maxima. Monotonicity is essential: since values only increase, every affected block maximum can be updated by taking a maximum with the new value.

Algorithm. Use a Fenwick tree with

$$s_i = \max_{j=f(i)}^{i} a_j.$$

The prefix query is the same right-to-left decomposition as in Problem 3, but using `max` instead of addition:

```
int getMaxPrefix(int pos) {
    int ans = -INF;
    for (int i = pos; i >= 0; i = f(i) - 1)
        ans = max(ans, s[i]);
    return ans;
}
```

For an update $a_{pos} \leftarrow a_{pos} + x$, first compute the new value $v = a_{pos}$, then update all blocks containing *pos*:

```
void increase(int pos, int x) {
    a[pos] += x;
    for (int i = pos; i < n; i = g(i))
        s[i] = max(s[i], a[pos]);
}
```

Correctness. The prefix query decomposes the prefix into disjoint Fenwick blocks. Since the maximum of a union of disjoint sets is the maximum of their block maxima, the query returns the correct prefix maximum.

After increasing a_{pos}, only Fenwick blocks containing *pos* can change their maximum. Every other block contains exactly the same values as before. For every block containing *pos*, the old maximum either remains the maximum or is replaced by the new value of a_{pos}. Thus assigning

$$s_i \leftarrow \max(s_i, a_{pos})$$

for exactly these blocks restores the invariant.

Running time. Initialization can be done by n Fenwick updates or by direct block construction, both within $O(n \log n)$ for the requested bound. Each query takes $O(\log n)$, so the total time is $O((n+q)\log n)$.

Problem 5. Longest increasing subsequence in $O(n \log n)$

Given $a_1, \ldots, a_n$ with pairwise distinct values, find a longest increasing subsequence in $O(n \log n)$.

Solution.

Idea. Dynamic programming says

$$dp_i = 1 + \max\{dp_j : j < i,\ a_j < a_i\}.$$

A Fenwick tree over compressed values can maintain this maximum over all previous smaller values.

Algorithm. Coordinate-compress the values of a so that $\text{rank}(a_i) \in \{1, \dots, n\}$ and

$$a_i < a_j \iff \text{rank}(a_i) < \text{rank}(a_j).$$

Maintain a Fenwick tree where each node stores a pair (ℓ, idx): the best LIS length ℓ ending at some processed element whose compressed value lies in the corresponding block, and the index idx of such an element.

Process $i = 1, 2, \dots, n$:

1. Query the maximum pair among ranks $1, \dots, \text{rank}(a_i) - 1$.
2. If the result is (ℓ, j), set $dp_i = \ell + 1$ and $parent_i = j$. If no smaller value has appeared, set $dp_i = 1$ and $parent_i = \perp$.
3. Update the Fenwick tree at position $\text{rank}(a_i)$ with the pair (dp_i, i), keeping the larger length.

At the end, take the index t with maximum dp_t and follow parent pointers backward; reversing this chain gives an LIS.

Correctness. When processing i, the Fenwick tree contains exactly information about indices $j < i$. Querying ranks smaller than $\text{rank}(a_i)$ therefore finds precisely the largest dp_j among all previous elements satisfying $a_j < a_i$. Hence the recurrence for dp_i is computed correctly. The stored parent realizes the chosen predecessor, so each parent chain is an increasing subsequence of the claimed length.

By induction on i, every dp_i equals the length of the longest increasing subsequence ending at i. Therefore the maximum dp_i over all endpoints is the global LIS length, and following the corresponding parent chain reconstructs an actual subsequence of that length.

Running time. Coordinate compression costs $O(n \log n)$. Each of the n steps performs one Fenwick maximum query and one update, both $O(\log n)$. The total time is $O(n \log n)$ and the memory usage is $O(n)$.

Problem 6. Point assignment and lower bound on prefix sums

A nonnegative array receives point assignments $a_{pos} := x$ and queries: given k, find the minimum r such that

$$a_1 + \cdots + a_r \geq k.$$

Process all queries in $O((n + q) \log n)$.

Solution.

Idea. A Fenwick tree maintains prefix sums under point updates. Since all values are nonnegative, prefix sums are monotone, so the desired r can be found by binary lifting inside the Fenwick tree.

Algorithm. Maintain a usual Fenwick tree for sums. For assignment $a_{pos} := x$, compute

$$\delta = x - a_{pos},$$

store $a_{pos} = x$, and apply the point update by δ.

To answer a query with parameter k, first check the total sum. If it is smaller than k, no such r exists. Otherwise find the largest index idx such that

$$a_1 + \cdots + a_{idx} < k.$$

This is done by descending bits from the largest power of two not exceeding n:

```
int lower_bound_prefix(long long k) {
    int idx = 0;
    long long sum = 0;
    for (int bit = highestPowerOfTwoAtMost(n); bit > 0; bit >>= 1) {
        int nxt = idx + bit;
        if (nxt <= n && sum + tree[nxt] < k) {
            idx = nxt;
            sum += tree[nxt];
        }
    }
    return idx + 1;
}
```

This pseudocode uses the common one-indexed Fenwick representation.

Correctness. The update correctness is the standard Fenwick invariant for prefix sums. For the query, after processing a bit, idx is the largest index with the already fixed high bits such that the accumulated prefix sum is still smaller than k. If adding the next Fenwick block keeps the sum below k, that bit must be set in the largest feasible index; otherwise setting it would overshoot or reach k, so the bit must remain zero. This greedy bit decision is valid because all array entries are nonnegative, hence prefix sums are monotone.

At termination, idx is the largest position with prefix sum $< k$. Therefore $idx + 1$ is the minimum position whose prefix sum is at least k.

Running time. Each assignment costs $O(\log n)$. The Fenwick lower-bound query tests $O(\log n)$ bits and costs $O(\log n)$. Thus all queries take $O((n + q) \log n)$.

Problem 7. Two-sided Fenwick tree for range maximum

For zero-indexed arrays define

$$s_i = \max_{j=f(i)}^{i} a_j, \qquad sr_i = \max_{j=i+1}^{g(i)} a_j.$$

Prove correctness of the given procedure for the maximum on $(l, r]$, and describe updates.

Solution.

Idea. The first loop decomposes a suffix of $(l, r]$ into ordinary Fenwick blocks ending at the current right endpoint. The second loop decomposes the remaining prefix into reverse Fenwick blocks starting immediately after the current left endpoint.

Algorithm. Use the query:

```
int getMax(int l, int r) {       // maximum on (l, r]
    int ans = -INF;
    while (f(r) > l) {
        ans = max(ans, s[r]);    // block [f(r), r]
        r = f(r) - 1;
    }
    while (g(l) <= r) {
        ans = max(ans, sr[l]);   // block (l, g(l)]
        l = g(l);
    }
    return ans;
}
```

For point increases, after setting a_{pos} to its new value v, update the direct blocks and reverse blocks:

```
for (int i = pos; i < n; i = g(i))
    s[i] = max(s[i], v);

for (int i = pos - 1; i >= 0; i = f(i) - 1)
    sr[i] = max(sr[i], v);
```

Correctness. Each iteration of the first loop uses the block $[f(r), r]$. The condition $f(r) > l$ guarantees that this block is fully contained in $(l, r]$. After adding it, replacing r by $f(r) - 1$ removes exactly that block from further consideration.

Each iteration of the second loop uses the block $(l, g(l)]$. The condition $g(l) \leq r$ guarantees that this block is fully contained in the remaining interval. Replacing l by $g(l)$ removes exactly this block.

It remains to justify that the two loops cover the whole interval. For any $l < r$, at least one of the two blocks $[f(r), r]$ and $(l, g(l)]$ is contained in $(l, r]$: otherwise $f(r) \leq l$ and $g(l) > r$ simultaneously, which contradicts the binary structure of Fenwick blocks at the first bit where l and r differ. Thus repeated greedy removal from the right and then from the left cannot leave a nonempty gap. The returned value is the maximum of the maxima of disjoint blocks whose union is exactly $(l, r]$.

For updates, the direct loop visits exactly the indices i such that $pos \in [f(i), i]$, and the reverse loop visits exactly the indices i such that $pos \in (i, g(i)]$. Therefore every stored block containing the changed point is updated and no other block is touched. Since updates are monotone increases, taking the maximum with the new value restores all block maxima.

Running time. Both query loops together visit $O(\log n)$ blocks. Each point increase also touches $O(\log n)$ direct and reverse blocks. Thus both operations take $O(\log n)$ time. For arbitrary decreasing assignments, these max Fenwick structures are not sufficient without recomputing block maxima or using a segment tree.

Problem 8. Range product modulo a prime using only a direct Fenwick tree

All computations are in $\mathbb{Z}/p\mathbb{Z}$ for a given prime p. Process point updates and range-product queries in

$$O((n+q)\log n + q\log p),$$

assuming modular inversion costs $O(\log p)$. Use only direct Fenwick trees.

Solution.

Idea. Division by a prefix product works only when no zero is present. Therefore store two direct Fenwick trees: one for the product of nonzero elements and one for the number of zeros.

Algorithm. Maintain

$$z_i = \begin{cases} 1, & a_i = 0, \\ 0, & a_i \neq 0, \end{cases}$$

and

$$b_i = \begin{cases} 1, & a_i = 0, \\ a_i, & a_i \neq 0. \end{cases}$$

Use one Fenwick tree for sums of z_i, and one multiplicative Fenwick tree for products of b_i modulo p.

For a point update $a_{pos} := x$:

1. Update the zero-count tree by $[x = 0] - [a_{pos} = 0]$.
2. In the product tree, remove the old nonzero factor if $a_{pos} \neq 0$ by multiplying by a_{pos}^{-1}, and insert the new nonzero factor if $x \neq 0$ by multiplying by x.
3. Store $a_{pos} = x$.

For a query on $[l, r]$, compute the number of zeros on the interval using the zero tree. If it is positive, return 0. Otherwise return

$$\frac{\prod_{i=1}^{r} b_i}{\prod_{i=1}^{l-1} b_i} \pmod p,$$

using one modular inverse for the denominator.

Correctness. The zero-count tree correctly reports whether the interval contains a zero. If it does, the interval product is 0 in $\mathbb{Z}/p\mathbb{Z}$. If it does not, then every element on the interval is nonzero, so all elements are invertible modulo the prime p, and the product on $[l, r]$ is exactly the quotient of two prefix products. The multiplicative Fenwick tree stores these prefix products over the modified values b_i, which agree with a_i on all nonzero positions.

Point updates preserve both invariants: the zero tree changes only at the updated position, and the product tree removes exactly the old nonzero contribution and adds exactly the new nonzero contribution.

Running time. Each Fenwick operation costs $O(\log n)$. Each update or nonzero query uses $O(1)$ modular inversions/multiplications costing $O(\log p)$ for inversions. The total time is $O((n + q)\log n + q\log p)$ and memory is $O(n)$.

Problem 9. Fenwick of Fenwicks for points in the plane

There are n points in the plane, each carrying a value a_i. Process point updates and queries for the sum of values in the rectangle $(0,0) \times (l,r)$, that is, among points with coordinates $x \le l$ and $y \le r$. Achieve $O((n+q)\log^2 n)$ time and $O(n \log n)$ memory.

Solution.

Idea. Build a Fenwick tree over the x-order of the points. Each outer Fenwick node stores an inner Fenwick tree over the y-coordinates of points in its canonical x-block.

Algorithm. Sort points by x-coordinate and give them ranks $0, 1, \ldots, n-1$. For every point of rank t, insert its y-coordinate into all outer Fenwick nodes reached by

$$t,\ g(t),\ g(g(t)), \ldots.$$

After all coordinates are collected, sort and deduplicate each node's list and build an inner Fenwick tree over that list.

A point update at point rank t with change δ updates all outer nodes on the same path $t, g(t), \ldots$; inside each such node, update the compressed position of the point's y-coordinate by δ.

To answer a prefix rectangle query $(x \le l, y \le r)$, let t be the largest rank with $x_t \le l$. Decompose the prefix $[0,t]$ by the usual Fenwick `get` loop. For every visited outer node, query its inner Fenwick tree for the prefix of y-coordinates at most r, and sum the answers.

Correctness. Each outer Fenwick node represents exactly one canonical block of consecutive x-ranks. The usual Fenwick prefix decomposition partitions all ranks with $x \le l$ into disjoint canonical blocks. For each such block, the inner Fenwick tree stores precisely the values of points in that block, indexed by y-coordinate. Its prefix query therefore returns exactly the contribution of points in that block with $y \le r$. Summing over the disjoint outer blocks gives exactly the sum over the rectangle.

For an update, a point belongs to exactly the outer blocks visited by the update path from its x-rank. Updating the corresponding inner Fenwick trees changes precisely all aggregate structures in which the point participates.

Running time. Each point is stored in $O(\log n)$ inner trees, so memory is $O(n \log n)$. Each update and each query visits $O(\log n)$ outer nodes and performs an $O(\log n)$ inner operation in each, giving $O(\log^2 n)$ time. Construction also fits within $O(n \log^2 n)$ time, and the requested total bound is $O((n+q)\log^2 n)$.

Problem 10. Range add and range sum in one dimension

Given $a_0, \ldots, a_{n-1}$, process range additions and range-sum queries in $O((n+q)\log n)$.

Solution.

Idea. Pass to the difference array

$$b_0 = a_0, \qquad b_i = a_i - a_{i-1} \quad (i \geq 1).$$

A range addition becomes two point updates in b. A range sum of a can be expressed through two prefix sums over b.

Algorithm. Maintain two Fenwick trees:

$$B(x) = \sum_{j=0}^{x} b_j, \qquad C(x) = \sum_{j=0}^{x} j b_j.$$

For range add $a_l, \ldots, a_r$ by val, do

$$b_l \mathrel{+}= val, \qquad b_{r+1} \mathrel{-}= val \quad \text{if } r+1 < n,$$

and apply corresponding point updates to both Fenwick trees B and C.

The prefix sum of the original array is

$$\sum_{i=0}^{x} a_i = \sum_{j=0}^{x} b_j (x+1-j) = (x+1) \sum_{j=0}^{x} b_j - \sum_{j=0}^{x} j b_j.$$

Thus define

$$P(x) = (x+1)B(x) - C(x).$$

Then the answer for $[l, r]$ is $P(r) - P(l-1)$, with $P(-1) = 0$.

Correctness. The difference-array identity gives $a_i = \sum_{j=0}^{i} b_j$. Hence summing over $i = 0, \ldots, x$ counts each b_j exactly $x - j + 1$ times, proving the formula for $P(x)$. A range addition changes the difference array only at the two boundary positions l and $r+1$, so the two point updates produce exactly the intended change in all a_i with $l \leq i \leq r$ and no change outside.

Therefore all range additions and range sums are represented correctly.

Running time. Each range addition performs $O(1)$ Fenwick point updates, and each range-sum query performs $O(1)$ Fenwick prefix queries. Every operation costs $O(\log n)$; total time is $O((n+q)\log n)$ and memory is $O(n)$.

Problem 11. Range add and range sum in two dimensions

Given an $n \times n$ table, process submatrix additions and submatrix-sum queries in $O((n^2+q)\log^2 n)$.

Solution.

Idea. Use the two-dimensional difference array

$$b_{ij} = a_{ij} - a_{i-1,j} - a_{i,j-1} + a_{i-1,j-1},$$

where missing indices contribute 0. A rectangle addition becomes four point updates in b. A prefix sum of a expands into four weighted prefix sums of b.

Algorithm. Use zero-indexing and define

$$X = x + 1, \qquad Y = y + 1.$$

Since

$$a_{ij} = \sum_{u=0}^{i} \sum_{v=0}^{j} b_{uv},$$

we have

$$\sum_{i=0}^{x} \sum_{j=0}^{y} a_{ij} = \sum_{u=0}^{x} \sum_{v=0}^{y} b_{uv}(x - u + 1)(y - v + 1).$$

Expanding gives

$$XY \sum b_{uv} - Y \sum u b_{uv} - X \sum v b_{uv} + \sum uv b_{uv},$$

where all sums are over $0 \le u \le x$, $0 \le v \le y$.

Maintain four two-dimensional Fenwick trees storing

$$b_{uv}, \qquad ub_{uv}, \qquad vb_{uv}, \qquad uvb_{uv}.$$

A rectangle addition by val on $[x_1, x_2] \times [y_1, y_2]$ applies the standard four updates to b:

(x_1, y_1)	$+val$
$(x_2 + 1, y_1)$	$-val$
$(x_1, y_2 + 1)$	$-val$
$(x_2 + 1, y_2 + 1)$	$+val$

ignoring points outside the table. Each such point update is inserted into all four weighted Fenwick trees.

A submatrix-sum query is answered by inclusion-exclusion from the prefix-sum function above.

Correctness. The two-dimensional difference identity implies that adding val to a rectangle changes b exactly at the four corners listed above. Thus after these updates, reconstructing a from b gives the intended modified table.

For a prefix rectangle $[0, x] \times [0, y]$, each difference value b_{uv} contributes to all cells a_{ij} with $u \le i \le x$ and $v \le j \le y$, which is exactly $(x - u + 1)(y - v + 1)$ cells. The expanded formula computes this weighted contribution using the four stored Fenwick aggregates. Inclusion-exclusion then converts arbitrary submatrix sums to four prefix sums.

Running time. Each rectangle update uses 4 difference-point updates, each updating 4 two-dimensional Fenwick trees; this is $O(\log^2 n)$ time up to constants. Each query uses 4 prefix queries, each reading 4 two-dimensional trees, also $O(\log^2 n)$. Initialization over n^2 cells fits the stated $O(n^2 \log^2 n)$ preprocessing bound, so the total is $O((n^2 + q) \log^2 n)$.

Problem 12. Range add and range sum in three dimensions

Given a table $a_{i,j,k}$ for $0 \le i, j, k < n$, process subcube additions and subcube-sum queries in $O((n^3 + q) \log^3 n)$.

Solution.

Idea. This is the exact three-dimensional analogue of Problem 11. Use a 3D difference array. Range addition becomes $2^3 = 8$ point updates. Prefix sums require $2^3 = 8$ weighted Fenwick trees.

Algorithm. Define the 3D difference array

$$\begin{aligned} c_{i,j,k} = a_{i,j,k} &- a_{i-1,j,k} - a_{i,j-1,k} - a_{i,j,k-1} \\ &+ a_{i-1,j-1,k} + a_{i-1,j,k-1} + a_{i,j-1,k-1} - a_{i-1,j-1,k-1}, \end{aligned}$$

where out-of-range terms are zero.

To add *val* to a box

$$[x_1, x_2] \times [y_1, y_2] \times [z_1, z_2],$$

update c at the eight corners

$$(x_1 \text{ or } x_2 + 1, y_1 \text{ or } y_2 + 1, z_1 \text{ or } z_2 + 1)$$

with sign $(-1)^t val$, where t is the number of upper endpoints chosen among $x_2 + 1, y_2 + 1, z_2 + 1$. Ignore corners outside the table.

For prefix sums, put $X = x + 1$, $Y = y + 1$, $Z = z + 1$. Then

$$\sum_{i=0}^{x} \sum_{j=0}^{y} \sum_{k=0}^{z} a_{i,j,k} = \sum_{u \le x, v \le y, w \le z} c_{u,v,w} (X - u)(Y - v)(Z - w).$$

Expand:

$$\begin{aligned} XYZ \sum c - YZ \sum uc - XZ \sum vc - XY \sum wc \\ + Z \sum uvc + Y \sum uwc + X \sum vwc - \sum uvwc. \end{aligned}$$

Maintain eight 3D Fenwick trees for the eight monomials

$$c,\ uc,\ vc,\ wc,\ uvc,\ uwc,\ vwc,\ uvwc.$$

Use the displayed formula for a prefix query, and use inclusion-exclusion over the eight corners of a query box to answer an arbitrary subcube sum.

Correctness. The 3D difference array satisfies

$$a_{i,j,k} = \sum_{u \le i} \sum_{v \le j} \sum_{w \le k} c_{u,v,w}.$$

Therefore a difference value $c_{u,v,w}$ contributes to exactly $(X - u)(Y - v)(Z - w)$ cells inside the prefix box. The eight weighted Fenwick trees compute the full expansion of this coefficient. Thus the prefix formula is correct, and ordinary 3D inclusion-exclusion gives arbitrary subcube sums.

For updates, the eight signed corner modifications are precisely the inverse operation to taking a 3D prefix sum. They make all points inside the requested subcube increase by *val* and make all exterior contributions cancel.

Running time. Each subcube update performs 8 point updates, each touching 8 Fenwick trees, each in $O(\log^3 n)$. A query similarly uses a constant number of 3D prefix computations. Hence every operation costs $O(\log^3 n)$ and preprocessing over n^3 values fits $O(n^3 \log^3 n)$, giving the required total $O((n^3 + q) \log^3 n)$.